

World Model DB: Paper Readiness Review

Sep 28, 2026 · @Shaunak J

Executive summary

World Model DB has a publishable idea but not yet a publishable paper: the semantics are distinctive, the time model has a correctness gap, and the evaluation does not exercise the core feature.

Verdict. RegattaDB is not the comparison that decides this paper. Regatta sells serializable, distributed current state for many agents; World Model DB sells explainable belief state. The reviewers' real comparison set is Zep/Graphiti, Mem0, Letta, XTDB and Datomic, plus the truth-discovery literature.

Thesis to own. Agents sharing memory need *epistemic state*: what is believed, since when, on what evidence, and against what contradictions. Existing agent memory resolves contradictions with non-reproducible LLM calls. World Model DB resolves them with a deterministic, versioned policy that can be replayed and asked *why*.

Top five actions.

1. Fix the time model: resolve per valid-time segment, let observations carry the valid time they claim, and declare predicate cardinality. Experiments show a plain CEO succession currently opens a conflict and late data produces inverted intervals.
2. Formalize the model and property-test replay determinism and order independence; today incremental ingest and rebuild disagree.
3. Build a conflict and staleness benchmark; this is the paper's strongest possible contribution.
4. Evaluate against Zep, Mem0 and a temporal-Postgres baseline, not against the project's own earlier build.
5. Ship a vision or workshop paper first; hold the full systems paper until concurrency and scale work lands.

What World Model DB is today

V0 is a clean, well-documented single-node prototype: about 6,700 lines of Rust across 12 crates, 31 passing tests, and one external runtime dependency (redb). The design documentation is unusually honest about its limits, which is a real asset for a paper.

Area	What exists	Assessment
Data model	Observations, facts, relationships, events, evidence, conflicts, correlations; five fact statuses	Strong conceptual core; the paper's formal model can be lifted from <code>docs/data-model.md</code>
Time	Half-open valid-time and known-time intervals, UTC RFC 3339 strings	Right idea; valid time is derived from the wrong field (see findings)
Resolution	Per (subject, predicate): group candidates by object, rank lexicographically, mark winner SUPPORTED, others CONTESTED	Deterministic and explainable, but assumes every predicate is single-valued and ignores valid-time segmentation
Provenance	Evidence links, <code>why</code> query returning observations, sources, conflicts	The most distinctive feature; nothing in the comparison set offers it
Agent layer	Agent registration as a source, idempotent <code>remember</code> , ranked context bundles with char budget	Good primitives; ranking is a fixed sort, not evaluated
Storage	All records held in memory as <code>Vec s</code> ; each save re-encodes and hashes every record, writes changed ones to redb	Correct but $O(N)$ per write, so ingest is $O(N^2)$ overall
Server	HTTP adapter with one global <code>Mutex</code> over the engine	Every request, including reads, is serialized
Tests	9 acceptance tests plus unit tests; all pass	Scenario tests only; no property tests, fuzzing or permutation tests
Benchmarks	14 workloads, 1,000 observations, 0% conflicts, single run, Windows	Micro-benchmarks of in-memory maps; no external baseline

In short: the semantics are the contribution, and the paper should lead with them. The storage engine is scaffolding and should be presented as such.

Comparison with RegattaDB

RegattaDB and World Model DB answer different questions: Regatta guarantees that concurrent agents see a correct *current* state; World Model DB explains *why* a state is believed and how that belief changed. They are complements, not rivals.

[RegattaDB](#) launched in July 2026 as a distributed SQL engine unifying OLTP, OLAP and vector search, marketed for "read-write AI" agents ([Blocks & Files](#), [Futurum](#)).

Dimension	RegattaDB	World Model DB V0
Core promise	Serializable consistency for many concurrent agents	Inspectable, reproducible belief state with evidence
Concurrency	Patented, mostly optimistic protocol; serializable without locks, snapshots or clock sync (architecture)	One global mutex; single writer
Scale	Claims thousands of nodes, 750k TPS on TPC-C (vendor figures)	One process, 1,000-observation benchmarks
Time	Internal multi-versioning; no user-facing time travel described	Valid time and known time as queryable dimensions
Provenance	Not described	Evidence links and why for every fact
Contradictions	Not a concept; the last committed write is the state	Preserved as conflict objects with statuses
Agent interface	MCP, Postgres-compatible SQL	REST, CLI, function-calling manifest; idempotent writes; context bundles
Research record	No peer-reviewed paper or independent benchmark found	None yet

What to borrow from Regatta.

- The framing of *read-write* agents: agents that act, not just retrieve. World Model DB should show what happens when agents write concurrently.
- Native MCP support as the default integration path.

- A clear single-sentence promise. Regatta's is "100% correct context"; World Model DB's should be "every belief is explainable and replayable".

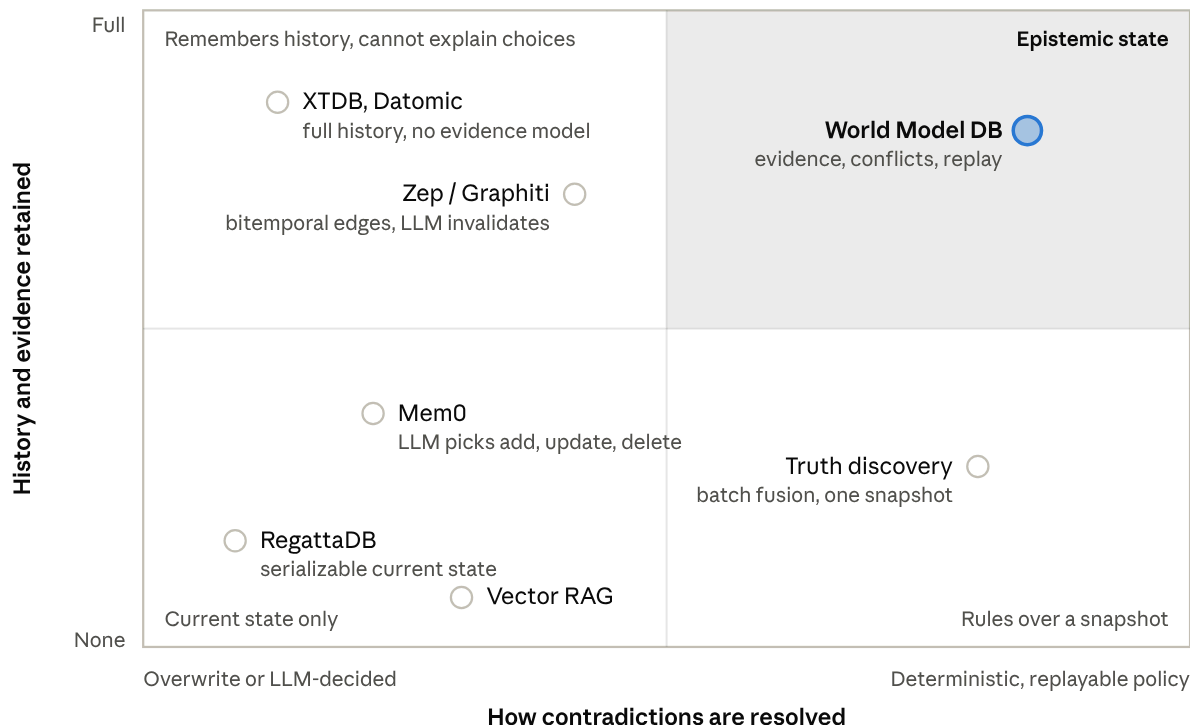
How to cite it in the paper. One sentence in related work: serializable engines such as RegattaDB guarantee consistent current state; epistemic state is orthogonal and could be layered over such an engine.

The real competitive landscape

The closest prior work is Zep/Graphiti, which already published a bitemporal knowledge graph for agent memory; the paper's novelty must be stated against it, not against Regatta.

Only World Model DB keeps full evidence and replays its decisions

Positioning by our reading of each system's published design



positioning map · 7 systems on evidence retained and resolution method

The empty upper-right quadrant is the paper's territory: every agent-memory system decides contradictions with an LLM call, and every bitemporal database stores history without an evidence or conflict model.

System	Time model	How contradictions are decided	Evidence kept	Evaluated on
Zep / Graphiti (Jan 2025)	Bitemporal edges: t_valid, t_invalid, t'_created, t'_expired	An LLM compares new edges with related ones; a contradiction sets the old edge's t_invalid	Episodes linked to edges; invalidated edges retained	DMR (94.8% vs MemGPT 93.4%), LongMemEval
Mem0 / Mem0g (Apr 2025)	Timestamps on memories	An LLM tool call picks ADD, UPDATE, DELETE or NOOP; Mem0g marks conflicting edges invalid	Partial	LoCoMo
Letta (MemGPT)	None explicit	The agent edits its own memory blocks	No	DMR
XTDB v2	SQL:2011 valid time and system time	Not a concept; last write per valid-time period	Full history, no evidence model	Not an agent system
Datomic	Transaction time, immutable log	Not a concept	Full history	Not an agent system
Truth discovery (TruthFinder, Knowledge Vault)	Usually none	Learned source reliability	Source links, batch snapshot	Accuracy vs gold facts
World Model DB	Valid time and known time on facts	Deterministic, versionable policy	Evidence links, conflicts, why	Micro-benchmarks only

What this means for the paper.

- The claim "bitemporal agent memory" is already taken by Zep. The new claims are *reproducible* resolution, *first-class* conflicts, and *explainable* belief through `why`.
- Zep and Mem0 are the required baselines. Their LLM-decided updates are the contrast: run each several times and show that decisions differ between runs, while World Model DB's do not.
- Truth discovery belongs in related work and as an optional resolver: a learned source-reliability model can sit behind the same interface.

Verified technical findings

Five of six experiments exposed a semantic defect that a reviewer could reproduce in minutes; all five trace to one root cause: resolution picks a single winner per (subject, predicate) across all of time. The existing 31 tests pass because none exercise these cases.

Method: built the repository at commit `656cf00` on Linux (Rust stable, release profile) and drove `wm-resolution` directly from a small harness. All timestamps below are the inputs used.

#	Experiment	Expected	Observed	Severity
E1	Alice CEO from 2020, Bob CEO from 2024, same filing source	Alice valid 2020–2024, Bob from 2024, no conflict	An open conflict; Alice’s current version is CONTESTED with an open-ended interval; "who was CEO in 2022?" with current knowledge returns nothing	Critical: the headline bitemporal query fails on a plain succession
E2	A September filing states Carol has been CEO since January	Carol valid from January	Carol valid from September; no field exists to state the claimed valid time	Critical: valid time is really "time the source spoke"

#	Experiment	Expected	Observed	Severity
E3	One source records two risks: "supply chain" and "currency"	Two facts, no conflict	"Supply chain" CONTESTED, one open conflict	High: every predicate is treated as single-valued; breaks agent memory tags and lists
E4	Observation ingested 2026-05-02, then a late one ingested 2026-04-02	Same state incrementally and after rebuild	Incremental: Dave gets valid [May 1, Apr 1) and known [May 2, Apr 2), i.e. inverted intervals , no conflict. Rebuild: a different state with an open conflict	Critical: contradicts the "deterministic reconstruction" claim
E5	Same source: Mumbai at 10:00+05:30 (04:30Z), London at 05:00Z	London supported (later in real time)	Incremental: London supported. After rebuild: Mumbai supported plus a conflict	High: offsets are accepted but compared as text
E6	Ingest cost as history grows (20% conflicting source)	Roughly constant per write	See table below	High for any scale claim

E6 ingest cost per observation. Each save re-encodes and hashes every record and resolution scans all facts, so cost grows faster than linearly.

Observations	Mean ms per write	Last 100 writes, ms each
500	2.7	4.1
1,000	5.2	10.0
2,000	11.9	26.4

Observations	Mean ms per write	Last 100 writes, ms each
4,000	33.0	79.7

At this rate a 100,000-observation store is impractical, so any paper scale experiment must wait for the storage fixes.

Other findings from code reading.

- The benchmark's "deterministic state digest" hashes observation and fact IDs, predicates and statuses, but not objects, valid or known intervals, or conflicts. E4's divergence in intervals would pass it unnoticed.
- `wm-server` holds the whole engine behind one `Arc<Mutex<Engine>>`, so reads and writes from all agents are serialized.
- The "confidence" stored on a fact is mean observation confidence plus a support bonus minus a conflict penalty; the winner is chosen by (priority, latest `observed_at`, summed confidence, count), so the stored number and the decision are only loosely related.
- Retracted facts get a zero-length valid interval `[t, t)`, so no point-in-time query can ever return them. Losing candidates are not affected; the earlier chat note saying they were was wrong.

The common fix is one design change: resolve per valid-time segment, not per (subject, predicate), with predicate cardinality declared in a schema. E1-E4 then become regression tests, and E4 becomes a permutation property test.

Improvement roadmap

Fix semantics first, then build the benchmark, then engineering; the first two are enough for a workshop or vision paper, and the third is needed only for a full systems paper. Effort figures are rough estimates for one engineer.

Priority	Area	Change	Fixes	Effort
P0	Time model	Add optional <code>claimed_valid_from</code> and <code>claimed_valid_to</code> to observations; default to <code>observed_at</code> only when absent	E2	2-3 days

Priority	Area	Change	Fixes	Effort
P0	Resolution	Resolve per valid-time segment: split the timeline at every claimed boundary and pick a winner per segment; a later claim closes an earlier one instead of contesting it	E1	1–2 weeks
P0	Schema	Declare predicate cardinality (one or many) and whether values are exclusive; multi-valued predicates never conflict	E3	3–4 days
P0	Time handling	Parse timestamps to UTC instants on ingest; store and compare as integers or normalized <code>Z</code> strings	E5	1–2 days
P0	Replay	Make incremental ingest equal to rebuild for any arrival order: on a late observation, re-resolve the affected key from scratch; assert <code>valid_from <= valid_to</code> and <code>known_from <= known_to</code>	E4	3–5 days
P0	Tests	Property tests: permutations of the same observations give identical canonical state; interval invariants always hold; digest covers objects, intervals and conflicts	E4, digest	3–4 days
P1	Policy	Persist <code>ResolutionConfig</code> with a version on every fact; add what-if replay that diffs two policies over the same history	Reproducibility claim	1 week
P1	Resolver	Present the rule order as an explicit policy language, and add one learned source-reliability resolver (TruthFinder-style) behind the same trait	"Brittle rules" objection	1–2 weeks

Priority	Area	Change	Fixes	Effort
P1	Naming	Rename <code>confidence</code> to <code>score</code> unless calibrated against gold data	Overclaiming	1 day
P1	Benchmark	Conflict and staleness generator with gold answers (section on evaluation)	The evaluation gap	2–3 weeks
P1	Agent interface	MCP server exposing remember, context, why, state-at	Adoption and baselines	3–5 days
P2	Storage	Keep history on disk; hold only indexes and current state in memory; incremental encode and hash only touched records	E6	2–3 weeks
P2	Writes	Batch and group-commit API; re-resolve only touched keys	E6	1 week
P2	Concurrency	Snapshot reads plus optimistic writes (<code>expected_version</code>) instead of a global mutex	Multi-agent claim	1–2 weeks
P2	Retrieval	Optional embedding index over entities and facts; every hit maps to a real ID	Memory benchmarks	1 week
P3	SQL	Temporal SQL through DataFusion once licensing allows	Usability	Later

The P0 rows are also the paper's formal core: once they land, the invariants they enforce become the theorems in the model section.

Paper positioning

Position the paper around one concept, *epistemic state*, and one sharp contrast: agent memory today resolves contradictions with LLM calls that cannot be replayed, while World Model DB resolves them with a versioned policy that can be replayed and asked why.

Thesis. Agents that share memory need more than current state. They need to know what is believed, since when, on what evidence, and against what contradictions. We make this *epistemic state* a first-class database abstraction and show it improves agent correctness at acceptable cost.

Why not "world model". In machine learning, a world model is a learned environment simulator (Dreamer, Genie). ML reviewers will expect that and be disappointed. Keep the project name; use "epistemic state" or "belief state" in the title and abstract.

Why not "agent-native database". That category is crowded (Zep, Mem0, Letta, and RegattaDB's own marketing). A category is not a contribution; a property is.

The three properties to own.

1. **Replayable.** The same observations and policy version always produce the same beliefs, in any arrival order.
2. **Explainable.** Every belief answers *why*: its observations, sources, competing candidates and the rule that decided.
3. **Contradiction-preserving.** Disagreement is data. Conflicts have a lifecycle and are visible to agents in their context.

Claims to avoid and what to say instead.

Avoid	Say instead	Why
"Database" in the systems sense, compared with Postgres or Regatta	"A storage and query layer for epistemic state"	V0 has no concurrency or scale story yet
"Confidence"	"Resolution score"	Not calibrated; the README already says so
"Truth" or "ground truth"	"Preferred belief"	The README's own disclaimer is the right tone
"YCSB" and "TPC" workload names	"Point-lookup microbenchmark"	The workloads read in-memory maps
36x speedup over V0 baseline	Nothing; drop it	Self-comparison carries no weight with reviewers
"Deterministic reconstruction" as a guarantee	State it only after the permutation property test passes	E4 currently disproves it

Framing RegattaDB and Zep in one line each.

- RegattaDB: guarantees consistent current state under concurrency; epistemic state is orthogonal and could run on top of it.
- Zep: introduced bitemporal agent memory; its invalidation is decided by an LLM and the losing evidence is not queryable as a conflict. We make resolution deterministic, versioned and explainable.

Contributions and paper outline

The paper should make four contributions; the benchmark is the one most likely to carry acceptance, because current memory benchmarks barely test contradicting sources or late corrections.

1. **Model.** A formal definition of epistemic state: observations with claimed valid time, candidates, facts over valid time and known time, conflicts with a lifecycle, and evidence links.

2. **Resolution.** A segment-wise, policy-versioned resolver with stated properties: replay determinism, independence from arrival order, monotone history (new evidence never rewrites what was known before), and what-if replay across policies.
3. **System.** An implementation with `why`, `changes`, world diff and temporal path queries, plus an agent interface: idempotent writes, conflict-aware context bundles within a budget, and MCP.
4. **Benchmark and evaluation.** A generator for changing, conflicting facts with gold answers, and results against Zep, Mem0, XTDB and a temporal-Postgres baseline.

The core definition, in one line. The belief about subject s and predicate p , valid at time t , as known at time k , is the policy applied to every observation known by k whose claimed interval covers t :

$$\mathrm{Belief}_{\pi}(s,p,t,k) = \pi\big(\{\, o \in O \mid o.s = s, \, o.p = p, \, o.\mathrm{ingested} \leq k, \, t \in [o.\mathrm{vfrom}, o.\mathrm{vto}) \, , \, \}\big)$$

Replay determinism then means the result depends only on the set O and the policy version π , never on arrival order. That is exactly the property E4 shows is currently violated, and exactly the theorem the paper should prove.

Title options.

- *Why Do We Believe That? Epistemic State as a Database Abstraction for Multi-Agent Systems*
- *Replayable Belief: Deterministic Conflict Resolution for Shared Agent Memory*
- *Beyond Memory: Bitemporal, Provenance-Complete Belief State for LLM Agents*

Abstract draft.

LLM agents increasingly read and write shared state, yet today's agent memory systems overwrite the past, resolve contradictions with non-reproducible LLM calls, and cannot explain why a fact is believed. We introduce epistemic state: immutable observations resolved into facts over valid time and known time by a versioned, deterministic policy, with conflicts and evidence retained as first-class objects. We formalize its invariants, implement it in World Model DB, and provide queries for provenance (WHY), change, world diff and temporal paths, plus an agent interface with idempotent writes and budgeted, conflict-aware context. On a new benchmark of conflicting, evolving facts, and on LongMemEval and LoCoMo, [results against Zep and Mem0]. Overhead relative to [XTDB / temporal Postgres] is [X].

Section skeleton (about 12 pages for a full paper, 4–6 for a workshop).

1. Introduction: a motivating multi-agent example (a research agent and a planner disagree about a company's CEO after a late correction); the three properties; contributions.
2. Background and related work: agent memory (Zep, Mem0, Letta), temporal databases (SQL:2011, XTDB, Datomic), provenance (semirings, why-provenance), truth discovery, HTAP engines such as RegattaDB.
3. Epistemic state model: definitions, fact and conflict lifecycles, invariants.
4. Resolution: segment-wise algorithm, policy language, properties with proofs or proof sketches, what-if replay.
5. System: storage, indexes, queries, agent interface, MCP.
6. Benchmark: generator design, parameters, gold answers, metrics.
7. Evaluation: correctness, reproducibility, agent tasks, cost and scale.
8. Limitations and future work: calibration, learned resolvers, distribution.
9. Conclusion.

Evaluation plan

The evaluation must exercise contradictions and change over time, which the current benchmark (0% conflicts) never does; four experiment families cover what reviewers will ask.

1. Epistemic correctness benchmark (new, the main contribution). A seeded generator producing entities, sources and ground-truth timelines, then noisy observations of them.

Knob	Range to sweep
Sources and their reliability	2–20 sources, reliability 0.5–0.99
Conflict rate	0%, 10%, 30%, 50% of keys
Change rate	Facts change 0–10 times over the timeline
Late arrival	0–40% of observations arrive after later ones
Backdating	Share of observations that claim a past valid time
Predicate cardinality	Mix of single- and multi-valued predicates

Metric	What it measures
As-of accuracy	Correct answer for (s, p, valid t) with current knowledge
Known-at accuracy	Correct answer for "what did we believe at k"
Conflict precision and recall	Real disagreements flagged; successions not flagged
Explanation accuracy	<code>why</code> names the observations that justify the answer
Replay variance	Answers changed across 5 runs and across shuffled arrival orders

2. Agent-memory benchmarks. LongMemEval (ICLR 2025; 500 questions over five abilities including knowledge updates and temporal reasoning) and LoCoMo, against Zep/Graphiti, Mem0 and a vector-store baseline, all with the same LLM. This requires an extraction step from chat into observations; report it separately so the storage contribution is not confused with extraction quality.

3. Multi-agent consistency. N agents (2, 8, 32) write and read shared facts with retries, handoffs and injected contradictions. Measure duplicate writes, lost updates, stale reads and time to see a correction. This is the only place RegattaDB-style concurrency matters, and it can be scoped to the single-node case.

4. Systems cost.

- Ingest rate, point-in-time query latency and `why` latency from 10^3 to 10^7 observations, at several conflict rates.
- Same hardware, Linux, at least 5 runs, median with 95% intervals.
- Baselines: XTDB v2 and PostgreSQL with system-versioned tables plus a provenance table, running the same queries.

Figures the paper needs.

1. Positioning map (as in this review).
2. Worked example: observations, candidates and facts on two time axes, with a `why` chain.
3. Fact and conflict lifecycle diagram.
4. Architecture.

5. As-of accuracy against conflict rate, per system.
6. Replay variance per system (the result that best makes the argument).
7. Scaling: ingest rate, query latency and memory against dataset size.

Hygiene to fix in the existing harness. Extend the state digest to cover objects, intervals and conflicts; drop YCSB and TPC names; report conflict rate and late-arrival rate in every result file.

Anticipated reviewer objections

Each likely objection has a concrete answer, but only if the matching experiment is in the paper; without it, each becomes a reject reason.

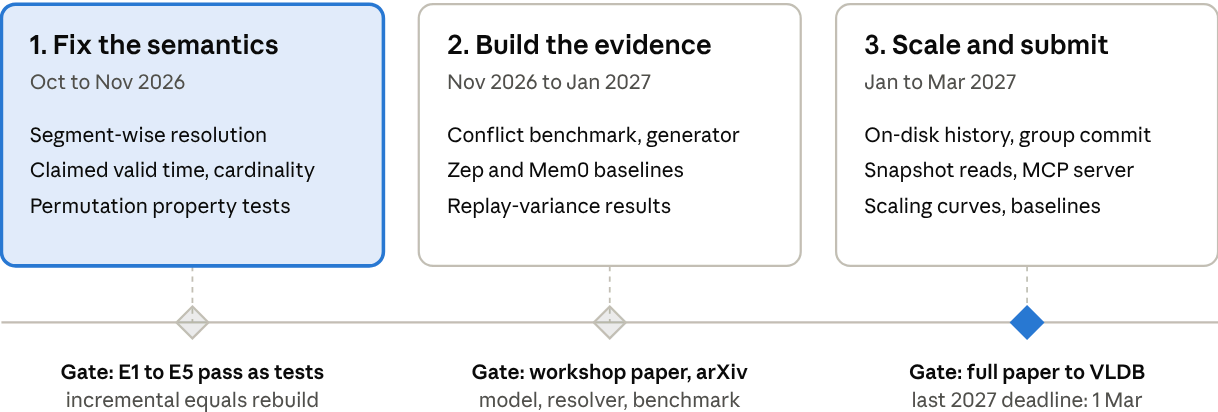
Objection	Response	Evidence needed
"This is XTDB plus a provenance table."	Build exactly that baseline and show what it cannot express cheaply: why at a past known time, conflict lifecycles, policy replay	Temporal-Postgres and XTDB baseline running the same queries
"Zep already does bitemporal agent memory."	Zep's invalidation is an LLM decision; it cannot be replayed and does not keep conflicts as queryable objects	Replay-variance figure; accuracy under injected contradictions
"Deterministic rules are brittle; LLMs handle nuance."	LLMs may propose candidates and entity matches; a versioned policy decides. Show a hybrid configuration	Hybrid run on the benchmark; learned-reliability resolver
"Does it help agents at all?"	End-to-end task over changing facts, with and without epistemic context	Task accuracy and stale-fact rate
"Your own benchmark favours your system."	Also report LongMemEval and LoCoMo; release the generator and seeds	Public generator, fixed seeds

Objection	Response	Evidence needed
"It doesn't scale."	Semantics are independent of the storage engine; show curves and state honest limits	Scaling curves after the P2 storage fixes
"Determinism claim is false with late data."	Currently true (E4). Fix it and prove it	Permutation property test and proof sketch
"Confidence is not a probability."	Agreed; it is a resolution score, or calibrate it	Rename, or a calibration plot

Venue strategy and timeline

Publish in two steps: a short paper on the model, resolver and benchmark first, then a full systems paper once scale and concurrency results exist. CIDR 2027's deadline (4 August 2026) has already passed, so the realistic targets are a workshop or arXiv preprint now and VLDB 2027 for the full paper.

Semantics first, evidence second, scale last



phased plan · 3 phases, 3 gates

Phase dates assume one engineer working steadily; each gate is the condition to move on, not a calendar promise.

Venue	Fit	Timing
arXiv preprint	Stakes the "epistemic state" claim early against fast-moving agent-memory work	Any time after phase 2
Data-management-for-AI workshops at SIGMOD or VLDB	Short paper on model, resolver and benchmark	Check each 2027 workshop's call
NeurIPS Datasets and Benchmarks track	If the conflict benchmark is strong enough to stand alone	Check the 2027 call
VLDB 2027 research track	Full systems paper	Monthly deadlines on the 1st, last one in March 2027; conference 23–27 August 2027, Athens
CIDR	Vision paper, 6 pages; welcomes risky architecture ideas	2027 deadline passed; watch for the 2028 call
Agent or memory workshops at NeurIPS, ICLR, ACL	Framed around agent correctness, not storage	Check each call

Message for Dharun

A short version to paste into chat; the sections above carry the detail.

Went through world-model-db in depth and compared it with RegattaDB. Summary:

Positioning. Regatta is a different class of system (distributed serializable OLTP/OLAP/vector; no provenance or time travel exposed). Our real comparison set is Zep/Graphiti (already published bitemporal agent memory), Mem0, XTDB and Datomic. Our angle should be *epistemic state*: what agents believe, since when, on what evidence, against what contradictions. The sharp claim is that Zep and Mem0 resolve contradictions with LLM calls that can't be replayed, while ours is a deterministic, versioned policy with WHY.

Bugs I reproduced (all 31 existing tests pass, none cover these):

1. A plain CEO succession (Alice 2020, Bob 2024, same source) opens a conflict, and "who was CEO in 2022?" returns nothing with current knowledge.
2. Valid time comes from `observed_at`, so a September filing saying "CEO since January" is valid from September.
3. Every predicate is single-valued: two different RISK memories from one agent become a conflict.
4. A late-arriving observation produces inverted intervals incrementally, and rebuild gives a different state, so the deterministic-reconstruction claim fails.
5. Timestamps with offsets are compared as strings; rebuild picks a different winner than incremental ingest.
6. Ingest is super-linear: about 5 ms per write at 1k observations, 80 ms at 4k, because every save re-encodes all records and resolution scans all facts.

Next steps.

- Phase 1: resolve per valid-time segment, add claimed valid time and predicate cardinality, normalize timestamps, and add permutation property tests.
- Phase 2: build a conflict and staleness benchmark with gold answers; run Zep and Mem0 against it and show replay variance.
- Phase 3: on-disk history, group commit, snapshot reads, MCP, and scaling curves.

Paper plan. Workshop paper or arXiv after phase 2 (model, resolver, benchmark); full paper to VLDB 2027 after phase 3. Drop the YCSB naming and the self-speedup numbers, and call confidence a score unless we calibrate it.

Sources

- [world-model-db repository](#), commit `656cf00`, built and tested on Linux for this review
- [RegattaDB home page](#)
- [Regatta's Architecture: A Bird's Eye View](#)
- [Blocks & Files: Regatta launches unified OLTP, OLAP and vector database](#)
- [Futurum: RegattaDB arrives](#)
- [Zep: A Temporal Knowledge Graph Architecture for Agent Memory \(arXiv 2501.13956\)](#)
- [Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory \(arXiv 2504.19413\)](#)
- [LongMemEval \(arXiv 2410.10813, ICLR 2025\)](#)
- [Time in XTDB](#)
- [CIDR 2027 call for papers](#)
- [VLDB 2027 important dates](#)